经验沉淀 · 华力集团研策部系统

如果重新从头搭建，这样走

基于 v1→v23 的真实踩坑经历，总结出的 8 步防返工路径。
每一步都对应一个实际发生过的问题。

为什么会反复修改？根本原因只有 3 个

在进入流程之前，先把这三件事刻进脑子里

01 需求没说完就动手了

用户边用边发现问题，AI 做一个改一个，功能完成后才发现边界条件。

「总监不作为被评分对象」→ AI 理解为「不出现在统计里」→ 实际是「不被别人打分，但自己负责的项目要统计」→ getOwners 改了 3 次

02 遇到模糊没追问，直接猜

「深蓝色」「一横一样」「按参考样式」—— AI 没停下来确认，靠理解直接做。

全局刷新按钮：垂直→横向→黄色→浅蓝→深蓝，改了 5 次才确定

03 改了一处，忘了同步其他地方

同一套逻辑在多处存在，改了主函数，忘了内联副本，导致显示不一致。

fmtMonDay 改了日期格式，内联的格式化代码没同步 → 同一页面两种日期格式并存

1

启动前

明确「给谁用、解决什么问题」

用自然语言回答三个问题：
① 使用场景是什么（谁、何时、在哪）
② 没有这个系统之前怎么做、哪里最痛
③ 有了之后什么东西变了

⚠ 坑：一开始就写周报表单，后来才发现要多人协作/看板/KPI。每个新需求都在打补丁

✅ 解法：先把场景过一遍。「当时觉得重要的功能」很多不重要，真正重要的一开始没想到

2

需求阶段

让 AI 反过来「审问」你

不是把需求告诉 AI，而是让 AI 问你：
- 多人同时用吗？
- 有权限区分吗？
- 填完之后要做什么？
- 有没有例外情况？

⚠ 坑：DIRECTORS 概念没定义清楚，三种理解都合理，代码只能实现一种

✅ 解法：AI 问问题的价值是「把没想到的边界条件逼出来」，而不只是收集信息

3

动手之前

先画数据结构，再写代码

把核心数据用最简单的 JSON 写出来，不需要完整，把关键字段列出来就够：

```
coreAction: ["事项1","事项2"]
// 永远是数组，不是字符串
designOwner: "冷耀秋"
// 可以是总监，不能过滤
```

⚠ 坑：coreAction 老数据是字符串，新数据是数组，混用写了兼容函数白费时间

✅ 解法：数据结构是地基，地基定了上面怎么建都可以改；地基歪了每一层都歪

4

技术选型

只选一次，选之前先问两个问题

① 国内用户能正常访问吗？（不能开 VPN）
② 这个技术三年后还在吗？

⚠ 坑：最开始用 Firebase，国内需要 VPN，整体迁移到 CloudBase，完整返工一次

✅ 解法：技术选型改一次 = 所有代码重写一次，这是成本最高的返工

5

第一版

最小可用版本，只做核心功能

列出所有想要的功能，问自己：
去掉哪些，这个系统还能用？
留下来的才是第一版要做的。

- ✅ 第一版：填写周报 · 云端保存 · 多人可看
- ○ 第二版：可视化看板
- ○ 第三版：KPI 评分
- ○ 第四版：年度统计

⚠ 坑：KPI 加进来时需要读周报数据 + 写评分 + 权限，每次都动核心数据层

✅ 解法：第一版的价值是验证方向，不是验证功能多不多

6

写代码前

建立「改动规范」放在项目里

在 RULES.md 里写三件事：
① 命名规范（函数/变量怎么命名）
② 数据格式约定（日期用 ISO 8601，不混用）
③ 修改规则（改逻辑必须全局搜索同类逻辑）

⚠ 坑：日期在不同地方有三种写法：YYYY-MM-DD / X月X日 / MMDD，统一改了两次

✅ 解法：规范是给「下一次修改」用的，防止每次改动引入新的不一致

7

每次改动前

先说清楚影响范围，再动手

每次提需求前先说：
「我要改 X，这会影响 Y 和 Z，帮我确认还有没有其他地方要同步。」

如果不知道影响范围，先问：
「这个功能涉及哪些地方？改之前帮我列一遍。」

⚠ 坑：改了主函数的日期格式，内联副本没同步 → 同一页面两种格式并存

✅ 解法：AI 只看到你给它的那段代码，它不知道「其他地方还有同样逻辑」除非你告诉它

8

每次部署前

过一遍 5 条部署清单

- ☐ 影响到的所有地方都改了吗？
- ☐ 数据格式有新旧不兼容的地方吗？
- ☐ 有没有写死人名 / URL / ID？
- ☐ 手机上看效果对吗？
- ☐ 旧数据能正常读取吗？

⚠ 坑：云函数 URL 曾写死成别人的地址，部署后才发现

✅ 解法：花 2 分钟过清单，省去 20 分钟线上返工

总结

这八步不是流程，是**防返工的护栏**。
每一步的存在都对应这个项目里实际发生过的一个坑。
走完一遍之后，这张图就是地图，下次可以少踩几个。

> *「每一层都需要踩到才能知道，*
> *这条路不走一遍是不知道坑在哪里的。」*